

Supply Chain Integration Hub — Demo Recording Script (Full)

Total target length: **9-10 minutes**. Read the SAY lines naturally — don't recite them word-for-word, use them as a guide so you never go blank mid-recording. Practice once silently before you hit record.

Before you record:

- Cmd+Shift+5 → Options → mic on, Show Mouse Clicks on → Record Entire Screen. Do Not Disturb on. Close anything with personal info in it.
- Have these ready in advance: (1) your app's Login page in the browser, (2) VS Code open with the backend already running in the integrated terminal (`./mvnw spring-boot:run`) — **start it fresh, right before you record, so the log starts from zero and is easy to scroll through later**.
- The System 2 simulator fires on its own timer in the background (`simulator.interval-ms` in `application.properties`) — with the current dev setting, it fires roughly every couple of minutes, so by the time you reach Scene 6 it will very likely have already fired once or twice on its own, with no action needed from you.
- Do this **before your SAP CPI trial expires** — everything in Scene 6 depends on the trial still being live.

Scene 1 — Intro (0:00–0:20)

[ON SCREEN] Desktop or Login page, nothing playing yet.

SAY: "Hi, this is a demo of my project — the Supply Chain Integration Hub. It's a full-stack purchase order and inventory system where two independent organizations, System 1 and System 2, exchange purchase orders, approvals, and inventory updates through SAP Cloud Platform Integration — SAP CPI — as the only bridge between them. System 1 is what I'll actually log into and click through today, since that's the human-operated side. I'll explain System 2 in a minute, and then prove the CPI integration is real by watching it happen live in the backend terminal."

Scene 2 — System 1: Procurement profile (0:20–2:20)

[ON SCREEN] Login page → Procurement tab.

SAY: "Let's start with the Procurement profile — one of the three logins on System 1."

[DO] Log in as Procurement.

SAY (on dashboard): "This is the Procurement dashboard. It shows this tenant's own order activity — nothing from the other organization leaks in here, everything's scoped to this login's tenant."

[DO] Navigate to Purchase Orders (Orders page).

SAY: "This is the Purchase Orders view. I can filter by tab — all orders, active, pending, cancelled — search by order ID or counterparty, and sort. Let's create a new one."

[DO] Click Create/New Order. Pick a vendor and a line item from the dropdown.

SAY (while filling the form): "Notice the line-item picker here — it's pulling live from the vendor's own inventory, so I'm only ever ordering things that actually exist on their side, not typing in arbitrary product names."

[DO] Submit the order.

SAY: "And it's created. Since I'm Procurement raising an order against a System 2 vendor, this gets dispatched to SAP CPI automatically the moment I hit submit — no separate 'send' button. I'll come back and prove that actually happened once I've shown you the other two profiles."

[DO] Quick glance at Suppliers page.

SAY: "Procurement also manages the supplier directory — company details, contact info, status — which is where that vendor dropdown pulls its list from."

[DO] Log out.

Scene 3 — System 1: Vendor profile (2:20–4:20)

[ON SCREEN] Login page → Vendor tab.

SAY: "Now the Vendor profile — the other side of a domestic order inside System 1."

[DO] Log in as Vendor.

SAY (on dashboard): "Vendor dashboard — again, scoped only to this tenant's outbound sales orders."

[DO] Navigate to Sales Orders.

SAY: "Here's my pending-approval queue. When an order comes in, I check stock and notify the buyer how much I actually have available."

[DO] Open a pending order, click Notify Availability / Notify Stock, enter a quantity.

SAY: "I'm telling the buyer I can supply this many units. Once they respond — either through the app or, in the cross-system case, back through CPI — I get to do the final confirmation, which is what actually kicks off the shipment lifecycle."

[DO] If a buyer-approved order is available, click Confirm Supply. Otherwise, just narrate it.

SAY: "Once confirmed, this order automatically starts moving through Processing, In Transit, and Delivered on its own — those are scheduled background transitions, not manual clicks."

[DO] Navigate to Inventory.

SAY: "This is the Vendor's inventory — stock levels, receive and sell operations, and a reorder threshold per item. If anything drops below its threshold, an alert fires automatically, and it also gets emailed out through another CPI flow."

[DO] Navigate to Shipments.

SAY: "And Shipments tracks the physical delivery side — pending, in transit, out for delivery, delivered — for everything this vendor has confirmed."

[DO] Log out.

Scene 4 — System 1: Admin profile (4:20–6:00)

[ON SCREEN] Login page → Admin tab.

SAY: "Last profile — Admin. This is the only login that can see across both tenants at once."

[DO] Log in as Admin.

SAY (on dashboard): "This is the cross-tenant view — combined order and inventory analytics that neither the Vendor nor Procurement login can see individually."

[DO] Navigate to the Vendor view and Procurement view (admin oversight pages).

SAY: "Admin also gets a direct oversight window into what the Vendor and Procurement dashboards look like, without needing to log in as them."

[DO] Navigate to User Management.

SAY: "And full user administration — locking, unlocking, and changing roles for any account."

[DO] Navigate to Alerts, then Integration Logs (don't dive deep into Integration Logs yet — save the detailed trace for Scene 6).

SAY: "I'll come back to this Integration Logs page in a minute — it's actually the most important page for proving the CPI side works, so I want to show you the raw backend evidence first."

Scene 5 — What System 2 actually is (6:00–6:45)

[ON SCREEN] Can stay on the Admin dashboard, or cut to a blank screen — this part is spoken, not clicked.

SAY: "Quick explanation before I show you the integration itself. System 2 isn't a second app with its own login screen — it's a simulator running inside the exact same backend process, but completely isolated by its own tenant ID, so its data never mixes with System 1's. On a scheduled timer, it automatically raises purchase orders addressed to System 1's vendor, and it automatically approves or occasionally rejects the orders System 1's procurement side sends out. The reason it exists is so I can demonstrate live, continuous, two-way traffic across the CPI bridge without needing a second person or a second deployed system on the other end. But it's not a shortcut — it goes through the exact same outbound code path and the exact same SAP CPI endpoint that a real, physically separate System 2 backend would use."

Scene 6 — Proving the CPI integration, from the terminal (6:45–9:00)

[ON SCREEN] Switch to VS Code, to the terminal where the backend has been running since before you started recording.

SAY: "Now let's prove this is a real integration, not a mocked one — and I'm not going to trigger anything new here, I'm just going to scroll back through this same terminal and show you what actually happened while I was clicking through the app a minute ago."

[DO] Scroll back up in the terminal to the point where you created the order in Scene 2. Find and read out loud a line like:

```
Order ORD-XXXXXXX sent to CPI (corrId=system1-ORD-XXXXXXX)  
CPI sendPo OK corrId=system1-ORD-XXXXXXX attempt=1
```

SAY: "So this is the exact moment I submitted that purchase order as Procurement. Order sent to CPI, then CPI sendPo OK with the correlation ID and the attempt number — that's my backend making a real, OAuth2-authenticated HTTP call out to my SAP CPI trial tenant and getting a successful response back. Nothing here is mocked or stubbed."

[DO] Scroll down toward the more recent end of the log. Find a line like:

```
Generator sent PO PO-S2-XXXXXX (xml) to System 1 vendor via iFlow1
iFlow1 PO received correlationId=system2-PO-S2-XXXXXX source=system2 target=system1
Saved inbound PO mirror orderId=PO-S2-XXXXXX correlationId=system2-PO-S2-XXXXXX
systemId=SYSTEM1 direction=OUTBOUND status=REQUESTED
```

SAY: "And here's the other direction, and I didn't trigger this one at all — this is System 2's simulator firing on its own scheduled timer while we were talking. It generated a purchase order, sent it out through the same iFlow 1, and my backend received it right back in on the inbound side, with a correlation ID tying the whole thing together. If I switched to the Vendor login right now, this exact order would be sitting in that pending-approvals queue I showed you earlier."

[DO — only if a stray debug-style log line is visible, e.g. tagged `iFlow4-trace` or a `JWT DEBUG` print]:

SAY (if needed, said lightly, not defensively): "You'll notice some extra diagnostic logging in here too — that's verbose trace logging I left in from debugging the inventory-update flow earlier, it's just internal instrumentation, not an error."

Scene 7 — Close (9:00–9:30)

[ON SCREEN] Back to VS Code or any dashboard.

SAY: "So that's the full picture — three role-based logins on System 1, a simulated System 2 counterparty, and a real, OAuth2-secured, retry-and-audit-hardened integration with SAP CPI proven straight from the backend logs, not just the UI. Spring Boot and MongoDB on the backend, React on the frontend, JWT auth throughout. Thanks for watching."

Quick reference — things to have ready before you hit record

- ☐ Procurement, Vendor, and Admin login credentials for System 1
- ☐ Restart the backend fresh (./mvnw spring-boot:run) right before you hit record, so the terminal log is short and easy to scroll through in Scene 6
- ☐ Keep the terminal window/tab easy to switch back to — you'll return to it once, in Scene 6, to scroll back through it
- ☐ Know roughly how long simulator.interval-ms is set to in your application.properties, so you're not surprised by when the simulator's own log lines show up